Understanding the Qwen-VL Preprocessing Pipeline

Chat Templates, Vision Extraction, and Processor Inputs

Contents

1 Overview **1**

2 `processor_qwen.apply_chat_template()` **1**

2.1 `tokenize=False` 2

2.2 `add_generation_prompt=True` 2

3 `process_vision_info(conversation)` **2**

4 `processor_qwen(...)` **2**

4.1 Text input 3

4.2 Image input 3

4.3 Video input 3

4.4 Padding and PyTorch tensors 3

5 Moving inputs to the GPU and generating **3**

6 End-to-end responsibility map **4**

7 Final conceptual clarification **4**

1 Overview

In Qwen vision-language models, a conversation may contain text, images, videos, or mixtures of these modalities. The model cannot directly consume an arbitrary Python conversation dictionary, image object, or ordinary text string. The input must be organized into the model's expected prompt structure and converted into numerical representations.

The three operations considered here have distinct responsibilities:

`apply_chat_template` ⟶ `process_vision_info` ⟶ `processor_qwen(...)`

The first formats the conversation, the second extracts/prepares visual data, and the processor tokenizes text and preprocesses the visual inputs.

2 `processor_qwen.apply_chat_template()`

A structured conversation typically contains messages with roles such as `system`, `user`, and `assistant`. Each message has content, which may include text and visual items.

For example:

```
conversation = [
    {
        "role": "user",
        "content": [
            {"type": "image", "image": image},
            {"type": "text", "text": "Describe this image."}
        ]
    }
]
```

Qwen expects a specific chat protocol. The template converts the structured conversation into a formatted prompt using role and modality markers. A conceptual text-only example is:

```
<|im_start|>user
Hello<|im_end|>
<|im_start|>assistant
```

Relevant markers can include `<|im_start|>` for a message start, `<|im_end|>` for a message end, and vision-related markers such as `<|vision_start|>` and `<|image_pad|>`. The exact representation depends on the model version and modality.

The common call is:

```
text = processor_qwen.apply_chat_template(
    conversation,
    tokenize=False,
    add_generation_prompt=True
)
```

2.1 `tokenize=False`

This is important. With `tokenize=False`, the function returns a formatted string, not token IDs. The readable special-token spellings are present in that string, but they have not yet been converted into integer token IDs.

If tokenization were requested here, the output would instead be integer IDs. In the pipeline discussed here, tokenization is deferred to the processor call.

2.2 `add_generation_prompt=True`

This appends the prefix indicating that the assistant's response should begin. Conceptually, the formatted prompt ends with:

```
<|im_start|>assistant
```

This is useful before calling `model_qwen.generate(...)`. The template formats the prompt; it does not itself run the model or generate a response.

3 `process_vision_info(conversation)`

This helper, commonly imported from `qwen_vl_utils`, extracts and prepares the actual visual content separately from the formatted prompt:

```
image_inputs, video_inputs = process_vision_info(conversation)
```

For a conversation containing two images and no video, the result is conceptually a collection of the two image inputs and no video input. For video, the helper may decode or prepare frames and associated information, depending on the input representation and library version.

The key distinction is that a visual marker in the formatted prompt is not the image itself. The template indicates where visual content belongs in the conversation; `process_vision_info` supplies the actual image/video data to the processor.

It is therefore not quite correct to say that this helper converts images into the final Qwen chat-template representation or fully converts them into model-ready patches. Its role is to extract and prepare the visual data. The processor performs the model-specific visual preprocessing.

4 `processor_qwen(...)`

The processor combines the formatted prompt and extracted visual inputs:

```
inputs = processor_qwen(
    text=[text],
    images=image_inputs,
    videos=video_inputs,
    padding=True,
    return_tensors="pt"
)
```

4.1 Text input

The `text=[text]` argument supplies the formatted prompt. The processor performs the main text tokenization in this pipeline, converting the string and special tokens into token IDs. It may also produce an attention mask.

4.2 Image input

The `images=image_inputs` argument supplies the actual images. The processor performs model-specific preprocessing, which can include resizing or resolution adjustment, conversion into numerical pixel tensors, and generation of image-grid metadata. The vision encoder subsequently uses these representations to produce visual features.

Image handling is not necessarily a simple fixed-size resize. Resolution and visual-token count can depend on the image and model configuration.

4.3 Video input

The `videos=video_inputs` argument supplies the prepared video content, commonly represented through frames and associated metadata. The processor converts these into the representations expected by the particular Qwen model. Exact temporal fields and output keys depend on the model and library versions.

4.4 Padding and PyTorch tensors

`padding=True` pads shorter sequences when needed so multiple examples can be represented in compatible batch shapes. An attention mask distinguishes real tokens from padding.

`return_tensors="pt"` requests PyTorch tensors. Depending on the modalities and implementation, the returned batch may include fields such as:

`input_ids`, `attention_mask`, `pixel_values`,

as well as image-grid or video-related metadata. The exact keys are model-dependent.

5 Moving inputs to the GPU and generating

A typical next step is:

```
inputs = inputs.to("cuda")
generated_ids = model_qwen.generate(
    **inputs,
    max_new_tokens=128
)
```

Moving inputs to `cuda` places the relevant tensors on the GPU; it does not perform additional preprocessing. The model must also be on a compatible device.

The expression `**inputs` unpacks the processor output as named model arguments. The `generate()` call performs inference and produces output token IDs, which can then be decoded into readable text.

6 End-to-end responsibility map

Operation	Responsibility
`apply_chat_template()`	Formats the entire conversation structure, including roles and appropriate modality markers. With `tokenize=False`, returns a string.
`process_vision_info()`	Extracts and prepares the actual image/video data separately.
`processor_qwen(...)`	Tokenizes formatted text and preprocesses image/video data into model-compatible tensors and metadata.
`model.generate()`	Runs generation using the prepared inputs.

7 Final conceptual clarification

The chat template does not handle only text: it formats the full conversation and places modality markers where required. The vision helper does not create the chat-template representation: it supplies the actual visual content. The processor brings the formatted text and visual data into the numerical representations expected by Qwen.

The concise sequence is:

Format conversation → extract visual data → tokenize and preprocess → run model.

The explanation above reflects the common Qwen2.5-VL-style pipeline. Qwen3-VL and different versions of Transformers or `qwen_vl_utils` may have additional options or different representations.